

FirstBank

DIGITAL FACTORY

TAKE-HOME TASK

NovaWallet Ledger Service

Backend Engineer — .NET / C#

Product scenario: FirstBank NovaPay

Issued: 24 August 2026

APPLICANT COPY

1. Scenario

FirstBank NovaPay — One super-app for everyday money — spend, save, borrow, collect, and send home.

FirstBank NovaPay is a fictional digital-first financial super-app built by FirstBank's Digital Factory for the Nigerian and West African mass market. It brings together five modules under one login: **NovaWallet** (e-wallet, P2P transfers, bill payments, airtime & data top-up), **NovaSave** (goal-based micro-savings, "Safe Lock" fixed savings, round-up savings), **NovaLend** (instant micro-loans / BNPL priced off alternative credit scoring built from wallet transaction history), **NovaBiz** (QR/POS-lite payment collection for small merchants), and a **diaspora remittance corridor** (inbound transfers from the UK, US and Canada landing directly in NGN wallets).

OPERATING CONTEXT

All work should assume a realistic Nigerian fintech operating environment: amounts in Naira (₦), stored in kobo to avoid floating-point drift; tiered KYC using BVN/NIN; instant settlement over NIBSS NIP rails; a USSD channel (*894#) for feature-phone / low-connectivity users; and compliance touchpoints with the CBN's consumer protection and licensing guidelines and the Nigeria Data Protection Act (NDPA 2023). Candidates are not expected to be regulatory experts — but strong candidates will notice where these constraints matter.

2. The Task: "NovaWallet Ledger Service"

Build a backend service in **C# / .NET (8 or 9)** that implements a simplified wallet ledger for the NovaWallet module. This is the component that must never lose, duplicate, or miscount a customer's money — treat it with the seriousness FirstBank would.

2.1 Functional requirements

CAPABILITY	REQUIREMENT
Create wallet	Create a wallet for a customer id; starting balance zero.
Get balance	Return current balance and currency (NGN), amounts in kobo.
Credit wallet	Deposit funds into a wallet (simulating an inbound NIP transfer).
Transfer	Move funds atomically from one wallet to another. Must be concurrency-safe : concurrent transfer requests against the same wallet must never allow the balance to go negative or allow a double-spend.
Idempotency	The transfer endpoint must accept an Idempotency-Key header. Replaying the same key must not double-process the transfer, and reusing a key with a different payload must be rejected.
Statement	Paginated transaction history for a wallet, newest first.
Daily limit	Enforce a server-side daily outbound transfer limit (e.g. ₦500,000/day) per wallet, reset at midnight WAT.
Audit log	Every balance mutation is recorded in an append-only, immutable audit trail (separate from the transaction table judges can query).

2.2 Hard constraints (non-negotiable)

- All monetary amounts are stored and computed as **integers in kobo** — no float/double anywhere in the money path.
- Balance must never go negative under any interleaving of concurrent requests.
- Endpoints are protected by a JWT bearer check (a simplified/mock issuer is fine — the point is the middleware and claims handling, not building a full auth server).
- Errors return a consistent, structured format (RFC 7807 Problem Details is recommended).
- The service and its datastore must start with a single command (`docker compose up`) — the panel will actually run it.

⚙️ AI USAGE REQUIREMENT

You are encouraged to use AI tools (Copilot, ChatGPT, Claude, Cursor, etc.) — this is expected and part of what we're assessing. Include a file `AI_USAGE.md` in your repo that documents: which tools you used and for what; 2–3 concrete prompts you gave and what came back; at least one specific case where the AI's output was **wrong, unsafe, or naive for a financial system** (e.g. a concurrency bug, a rounding/precision issue, a missing edge case) and how you caught and fixed it. We are testing judgment in directing AI, not whether you avoided it.

2.3 Deliverables

- Link to a Git repository (GitHub/GitLab). If private, grant access to the emails provided in your interview invite.
- `README.md` — architecture & key decisions, trade-offs, how to run and test.
- `AI_USAGE.md` as described above.
- OpenAPI/Swagger spec reachable when the service is running.
- Automated tests, including at least one that exercises the concurrency edge case under load.

2.4 Stretch goals (optional — used to differentiate strong candidates)

- Rate-limiting middleware on the transfer endpoint.
- Outbox pattern publishing a `TransferCompleted` event.
- Structured logging with correlation/trace IDs across a request.
- Health/readiness endpoints suitable for container orchestration.

3. How you'll be assessed

CRITERION	WHAT WE'RE LOOKING FOR
Correctness & concurrency safety	The hard constraints in 2.2 actually hold under concurrent load, not just in the happy path.
Code quality & architecture	Clear separation of concerns, sensible use of .NET/C# idioms, maintainable structure.
Test rigor	Tests that would actually catch a regression, including the concurrency case.
Security awareness	Sensible handling of auth, input validation, secrets.
AI fluency & judgment	Evidence of using AI to move faster <i>and</i> evidence of catching where it was wrong.
Communication	Clarity of README and your ability to explain and defend decisions live.

4. Logistics

- **Time window:** 48–72 hours from receipt of this document to submission.
- **On interview day:** bring the repo up on a share screen, walk the panel through your design, and be ready to modify code live if asked.
- **Scope note:** this is intentionally more than can be "gold-plated" in the time given — we'd rather see good judgment about what to prioritize than a rushed attempt at everything.
- Questions about the brief may be sent to your recruiting contact; use your own judgment for anything left ambiguous and document the assumption in your README.

This is a fictional scenario created for evaluation purposes only. No real FirstBank systems, data, or credentials are involved.